Gibbs Diffusion for Blind Denoising

Generative Modeling for the Blind Denoising and Simultaneous Noise Parameter Estimation

A Report Presented
by
Alik Mondal

Departments

Department of Applied Mathematics and Theoretical Physics
Department of Physics (Cavendish Laboratory)
Kavli Institute of Cosmology

Degree

MPhil in Data Intensive Science

Supervision

Dr Boris Bolliet
Dr Fiona McCarthy

Girton College
University of Cambridge

July 7, 2025

Abstract

Blind denoising, the task of recovering a clean signal and its noise characteristics from a noisy observation without prior knowledge of the noise—remains a critical challenge in data analysis. This project investigates **Gibbs Diffusion (GDiff)**, a novel Bayesian framework proposed by Heurtel-Depeiges et al. (2024), which combines deep generative diffusion models with Gibbs sampling to address this problem. The GDiff methodology iteratively samples from the joint posterior of the signal and parametric Gaussian noise parameters by alternating between (i) denoising the signal using a conditional diffusion model given the current noise estimate, and (ii) inferring noise parameters via Hamiltonian Monte Carlo, given the current signal estimate.

This work focuses on the reproduction and analysis of the GDiff algorithm. Key findings from the original paper were successfully reproduced for two distinct applications: blind denoising of natural images corrupted by colored noise, and cosmological parameter inference from simulated Cosmic Microwave Background data. Quantitative metrics—including Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM), and Mean Absolute Error (L1) for image denoising, as well as posterior accuracy for cosmological parameters—are presented. The study confirms GDiff's ability to provide robust signal reconstruction and noise parameter estimation, while also highlighting computational considerations.

List of Figures

List of Tables

Contents

Total word count (excluding Contents, Figures, Tables and Bibliography): 6,103

Chapter 1

Introduction

1.1 The Ubiquity of Noise and the Denoising Challenge

Signal processing, at its core, is the discipline of extracting meaningful information from observed data. In virtually every real-world scenario, from the faintest astronomical signals reaching Earth-bound telescopes to the subtle electrical potentials recorded in biomedical diagnostics, and the fluctuating prices in financial markets, the desired underlying signal is invariably corrupted by unwanted disturbances, collectively termed 'noise' [1, 2]. The presence of noise can obscure critical features, lead to erroneous interpretations, and degrade the performance of subsequent analysis or decision-making processes. Therefore, the ability to effectively mitigate or remove noise is a cornerstone of modern data analysis and scientific discovery.

Noise can be broadly defined as any undesirable random or systematic component that interferes with the true signal of interest. It can originate from a multitude of sources, including thermal agitation in electronic components (thermal noise), quantization errors in digital systems (quantization noise), interference from external electromagnetic sources, or even inherent stochasticity within the phenomenon being observed (e.g., photon shot noise in imaging). Noise characteristics vary widely: it can be additive or multiplicative, white (uncorrelated with a flat power spectrum) or colored (exhibiting frequency-dependent correlations), Gaussian or non-Gaussian in its statistical distribution [3]. Understanding the nature of noise is often crucial for designing effective mitigation strategies.

The denoising problem aims to estimate the unobserved clean signal, denoted as x, from a noisy observation, y. A common model for this is the additive noise model, $y = x + \varepsilon$, where ε represents the noise component. The challenge lies in separating x from ε with minimal distortion to x and maximal suppression of ε. This task is inherently ill-posed, as there are infinitely many combinations of signal and noise that could produce the same observation. Consequently, effective denoising typically requires incorporating some form of prior knowledge or assumptions about the characteristics of either the signal, the noise, or both.

1.2 Traditional Denoising Approaches and Their Limitations

Historically, a plethora of methods have been developed to tackle the denoising problem, often relying on statistical models or transform-domain properties. Methods requiring known noise characteristics have achieved considerable success when such information is available. For instance, the Wiener filter, an optimal linear filter in the mean squared error sense, requires knowledge of the power spectral densities of both the signal and the noise [4]. In the transform domain, wavelet thresholding techniques have proven highly effective, particularly for signals that are sparse in the wavelet domain [5]. These methods typically involve decomposing the noisy signal into wavelet coefficients, attenuating or zeroing out coefficients likely dominated by noise (based on a threshold often derived from the noise variance), and then reconstructing the signal. However, the efficacy of these approaches hinges on accurate knowledge of noise properties, such as its variance or power spectrum, which may not always be readily available.

The scenario where noise characteristics are unknown presents the more challenging blind denoising problem. Early attempts at blind denoising often involved iterative estimation schemes, subspace methods based on singular value decomposition (e.g., for white noise removal if the signal has a low-rank structure), or methods that tried to estimate noise statistics from presumed signal-free regions of the data [6]. While some success was achieved, these methods often had limitations in their applicability to diverse signal types or complex, structured noise. They also frequently struggled with the inherent ambiguity of attributing signal variations to either the true signal or the noise process without strong priors.

1.3 The Deep Learning Revolution in Signal Processing

The advent of deep learning has brought about a paradigm shift in many areas of signal and image processing, including denoising [7, 8]. Convolutional Neural Networks (CNNs), in particular, have demonstrated remarkable capabilities in learning complex mappings from noisy to clean signals directly from data, often outperforming traditional methods.

CNN-based denoisers, such as the influential DnCNN (Denoising Convolutional Neural Network) architecture [9], are trained in a supervised manner on large datasets of noisy/clean signal pairs. These networks learn to implicitly capture the statistical properties of clean signals and the noise characteristics present in the training data. By leveraging hierarchical feature extraction, CNNs can effectively model non-local dependencies and adapt to varying signal structures, leading to state-of-the-art performance, especially for additive white Gaussian noise (AWGN) at known levels. Variations have also been developed to handle more complex noise types, though often still requiring the noise model to be fixed or known during training.

Beyond discriminative models like DnCNN, **generative models** have emerged as powerful tools for learning the underlying probability distribution of data, $p(x)$. Models like Generative Adversarial Networks (GANs) [10] and Variational Autoencoders (VAEs) [11] can learn to synthesize realistic-looking signals. In the context of denoising, these models can be used to define strong priors on the signal space. Denoising can then be framed as finding a clean signal from this learned prior that is consistent with the noisy observation, often through optimization or Bayesian inference techniques [12]. While powerful, training GANs can be unstable, and VAEs often produce overly smooth reconstructions.

1.4 Diffusion Models: A New Paradigm for Generative Modeling and Denoising

More recently, diffusion probabilistic models (DDPMs) [13, 14] and score-based generative models [15, 16] have emerged as a highly successful class of generative models, achieving state-of-the-art results in image synthesis, audio generation, and other domains. These models define a forward diffusion process that gradually adds noise to data, transforming it into an unstructured noise distribution (typically Gaussian). A neural network, often a U-Net architecture, is then trained to reverse this process by learning to predict the noise added at each step (in DDPMs) or the score function (gradient of the log probability density) of the noisy data distribution (in score-based models).

Once trained, these models can generate new samples by starting from pure noise and iteratively applying the learned reverse transformation. Crucially, for denoising, diffusion models inherently learn a powerful prior over the data distribution $p(x)$. If the noise process corrupting an observation y is known and matches (or can be related to) the forward diffusion process, the trained reverse process can be directly used to denoise y and sample from the posterior $p(x|y)$ [16, 17]. This capability to act as both a strong generative prior and a conditional sampler makes them exceptionally well-suited for inverse problems, including denoising under known noise conditions. Their ability to produce high-fidelity samples and capture fine details often surpasses that of earlier generative models.

1.5 The Blind Denoising Problem with Diffusion Models

Despite the power of diffusion models as conditional generators $p(x|y)$ when the noise characteristics are known, a significant gap remains: standard diffusion model-based denoising techniques typically assume that the noise level and covariance structure (if not simply AWGN) are provided as input to the reverse process. In many practical scenarios, this information is precisely what is unknown, constituting the blind denoising problem. Directly applying a diffusion model trained for a specific noise type or level to data with different, unknown noise will likely yield suboptimal or incorrect results.

The paper "Listening to the Noise: Blind Denoising with Gibbs Diffusion" by Heurtel-Depeiges et al. (2024) [18] (hereafter referred to as the GDiff paper) addresses this critical gap. It introduces Gibbs Diffusion (GDiff), a novel Bayesian framework that elegantly combines the generative power of conditional diffusion models with the inferential capabilities of Gibbs sampling. GDiff aims to simultaneously infer both the underlying clean signal x and the parameters ϕ characterizing an assumed parametric Gaussian noise model $\varepsilon \sim \mathcal{N}(0, \Sigma_\phi)$, given a noisy observation y. This is achieved by iteratively sampling from the full joint posterior $p(x, \phi|y)$.

1.6 Project Objectives and Contributions

This MPhil project is primarily a reproduction study focused on the Gibbs Diffusion methodology presented in Heurtel-Depeiges et al. (2024) [18]. The principal objective is to implement, understand, and validate the GDiff algorithm by replicating its key findings across the diverse applications showcased in the original work.

The secondary objectives of this project are:

- To gain a deep understanding of the theoretical underpinnings of GDiff, including its Bayesian formulation, the role of the conditional diffusion model, and the noise parameter inference step using Hamiltonian Monte Carlo (HMC).
- To quantitatively evaluate the performance of the reproduced GDiff implementation on benchmark tasks, specifically the blind denoising of natural images with colored noise and cosmological parameter inference from simulated Cosmic Microwave Background (CMB) data.

- To compare the GDiff model against benchmark methods such as BM3D under the non-blind denoising setting.
- To identify potential limitations, computational challenges, and areas where the GDiff framework could be further explored or extended.

While this work does not aim to introduce novel methodological extensions to GDiff, the process of reproduction, in-depth analysis, and critical evaluation constitutes a valuable scientific contribution by verifying and potentially providing new insights into a recently proposed advanced machine learning technique.

1.7 Project Outline

The remainder of this project report is structured as follows:

Chapter 2: Theoretical Background and Methodology provides a detailed review of the necessary theoretical concepts, including the Bayesian framework for denoising, diffusion probabilistic models, Gibbs sampling, and Hamiltonian Monte Carlo. It then elaborates on the GDiff algorithm itself, its components, and its theoretical properties. Implementation details relevant to this reproduction study, including datasets, model architectures, and evaluation metrics, are also presented.

Chapter 3: Results of GDiff Reproduction presents the core findings of this work. It details the outcomes of applying the GDiff implementation to the two main applications from the original paper: blind denoising of natural images and cosmological parameter inference. Qualitative and quantitative results are provided, directly comparing them with those reported by Heurtel-Depeiges et al. (2024).

Chapter 4: Discussion analyzes the reproduced results in a broader context. It discusses the strengths and weaknesses of the GDiff approach as observed during this study, highlights any challenges encountered during the reproduction process, and reflects on the implications of the findings.

Chapter 5: Conclusion summarizes the main understanding and conclusions of this project.

Chapter 2

Theoretical Background and Methodology

This chapter lays the theoretical groundwork necessary to understand the Gibbs Diffusion (GDiff) framework. We begin by formalizing the blind denoising problem within a Bayesian context. Subsequently, we review Diffusion Probabilistic Models (DDPMs) and their application to conditional signal generation. We then discuss the principles of Gibbs sampling for Bayesian inference, followed by an exposition of Hamiltonian Monte Carlo (HMC) for sampling complex distributions. Finally, we synthesize these components to detail the GDiff algorithm as presented by Heurtel-Depeiges et al. (2024) [18], and discuss implementation specifics relevant to this reproduction study, including datasets, model architectures, and evaluation metrics.

2.1 Bayesian Framework for Denoising

The problem of denoising can be elegantly formulated within a Bayesian statistical framework. We consider an observed noisy signal $y \in R^d$ which is assumed to be an additive mixture of an unknown clean signal $x \in R^d$ and an unknown noise realization $\varepsilon \in R^d$:

$$y = x + \varepsilon. \tag{2.1}$$

In the context of blind denoising with GDiff, the noise ε is assumed to follow a parametric Gaussian distribution with zero mean and covariance matrix Σ_ϕ, where $\phi \in R^K$ are unknown noise parameters:

$$\varepsilon \sim \mathcal{N}(0, \Sigma_\phi). \tag{2.2}$$

The functional form of the covariance Σ_ϕ in this project is taken to be diagonal in Fourier space for colored noise, parameterized by either by spectral index and amplitude or cosmological parameters depending on the problem setting.

The Bayesian approach seeks to infer the posterior probability distribution of the unknowns (x and ϕ) given the observation y. This is achieved by combining prior beliefs about the signal and noise parameters with the likelihood of observing y.

- **Prior Distributions:** We define a prior distribution $p(x)$ over the clean signal x, which encapsulates our assumptions or knowledge about the class of signals we expect (natural images and Interstellar dust). In GDiff, this prior is implicitly learned by a diffusion model. We also define a prior distribution $p(\phi)$ over the noise parameters, representing our initial beliefs about their plausible ranges or distributions.

- **Likelihood:** Given x and ϕ, the likelihood of observing y is derived from the noise model. Since $y - x = \varepsilon$ and $\varepsilon \sim \mathcal{N}(0, \Sigma_\phi)$, the likelihood is:

$$p(y|x,\phi) = \mathcal{N}(y; x, \Sigma_\phi) = \frac{1}{\sqrt{(2\pi)^d |\Sigma_\phi|}} \exp\left(-\frac{1}{2}(y-x)^T \Sigma_\phi^{-1}(y-x)\right). \tag{2.3}$$

- **Posterior Distribution:** Using Bayes' theorem, the joint posterior distribution of the signal x and noise parameters ϕ given the observation y is:

$$p(x,\phi|y) = \frac{p(y|x,\phi)p(x)p(\phi)}{p(y)}, \tag{2.4}$$

  where $p(y) = p(y|x,\phi)p(x)p(\phi)\, dx\, d\phi$ is the marginal likelihood or evidence, which acts as a normalizing constant.

The goal of blind denoising in this framework is to characterize or draw samples from this joint posterior $p(x,\phi|y)$. Since $p(x)$ is often intractable (implicitly defined by a deep network) and the integral for $p(y)$ is usually high-dimensional and difficult to compute, direct sampling or evaluation of the posterior is challenging, necessitating methods like Gibbs sampling.

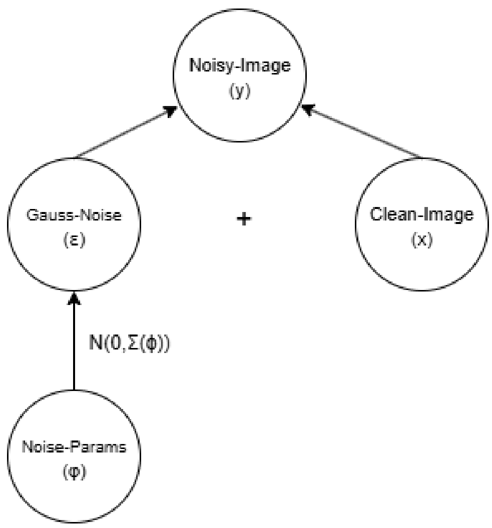


Figure 2.1: A graphical model illustrating the Bayesian framework for blind denoising. The observed data y depends on the true signal x and the noise ε. The noise ε is characterized by parameters ϕ. Our goal is to infer x and ϕ from y, given priors $p(x)$ and $p(\phi)$.

2.2 Diffusion Probabilistic Models

Diffusion Probabilistic Models (DDPMs) [13, 14] and their continuous-time counterparts, score-based generative models via Stochastic Differential Equations (SDEs) [16], have emerged as a powerful class of deep generative models. They learn to synthesize data by reversing a predefined noise-corruption process.

2.2.1 Forward Process (Noising)

The forward process gradually perturbs data $x_0 \sim p_{data}(x)$ (where x_0 is a clean signal, denoted x in other sections) over a sequence of T timesteps (in the discrete case) or a continuous time interval $t \in [0, T_f]$ (in the SDE case). At each step, a small amount of noise (typically Gaussian) is added. For a discrete-time DDPM, this can be written as:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I), \tag{2.5}$$

where $\{\beta_t\}_{t=1}^T$ is a predefined variance schedule. A key property is that x_t can be sampled directly from x_0:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I), \tag{2.6}$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. As $t \to T$, $p(x_T)$ approaches a standard Gaussian distribution $\mathcal{N}(0, I)$ if the schedule is chosen appropriately.

In the continuous-time SDE formulation, the forward process is often described by an Itô SDE:

$$dx_t = f(x_t, t)dt + g(t)dw_t, \quad t \in [0, T_f], \tag{2.7}$$

where w_t is a standard Wiener process, and $f(x_t, t)$ and $g(t)$ are drift and diffusion coefficients, respectively, designed such that $p_{T_f}(x)$ is a simple noise distribution. A common choice is the Variance Preserving (VP) SDE, which corresponds to the DDPM limit.

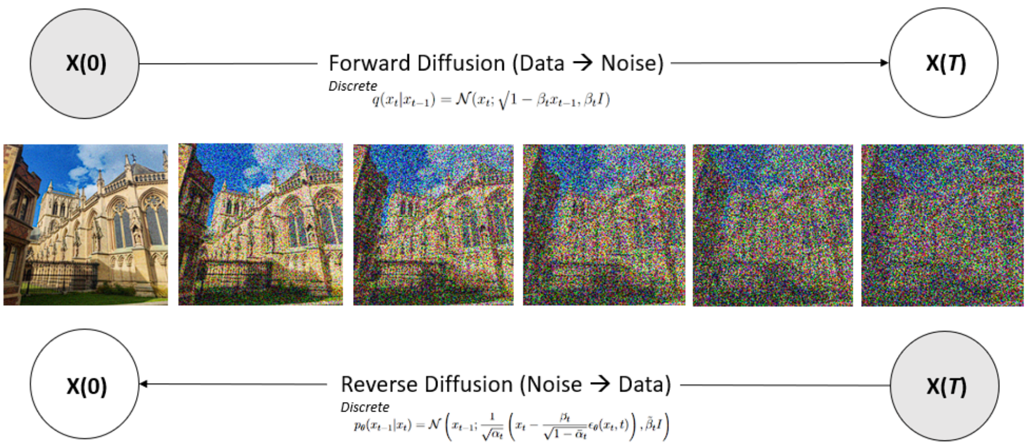


Figure 2.2: Schematic of the forward noising process and the learned reverse denoising process in a diffusion model. The forward process gradually adds noise to a data sample x_0, transforming it into $x_T \approx \mathcal{N}(0, I)$. The reverse process learns to denoise x_t at each step to recover x_0.

2.2.2 Reverse Process (Denoising/Generation)

The generative power of diffusion models lies in learning to reverse this forward noising process. The reverse process $q(x_{t-1}|x_t)$ is also Gaussian if β_t is small. For DDPMs, the mean of $p_\theta(x_{t-1}|x_t)$ is parameterized by a neural network $\epsilon_\theta(x_t, t)$ trained to predict the noise ϵ added to obtain x_t from x_0 (using Eq. 2.6, $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$).

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}\left(x_{t-1}; \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right), \tilde{\beta}_t I\right), \tag{2.8}$$

where $\tilde{\beta}_t$ is a chosen variance for the reverse step.

For SDEs, the reverse process is also an SDE that runs backward in time from $t = T_f$ to $t = 0$:

$$dx_t = [f(x_t, t) - g(t)^2 \nabla_{x_t} \log p_t(x_t)]dt + g(t)d\bar{w}_t, \tag{2.9}$$

where $\bar{w}_t$ is a standard Wiener process when time flows backwards, and $\nabla_{x_t} \log p_t(x_t)$ is the score function of the marginal distribution $p_t(x_t)$. A neural network $s_\theta(x_t, t)$ is trained to approximate this score function.

2.2.3 Training: Noise Prediction

The neural network $\epsilon_\theta(x_t, t)$ (for DDPMs) or $s_\theta(x_t, t)$ (for SDEs) is typically trained using a simplified objective. For DDPMs, a common loss is:

$$L_{DDPM}(\theta) = E_{t,x_0,\epsilon}\left[||\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)||^2\right], \tag{2.10}$$

where t is uniformly sampled from $\{1, ..., T\}$, $x_0 \sim p_{data}(x)$, and $\epsilon \sim \mathcal{N}(0, I)$.

2.2.4 Conditional Denoising with Known Noise

Diffusion models can be adapted for conditional denoising $p(x \mid c)$ by providing the conditioning information c as an additional input to the network, i.e., $\epsilon_\theta(x_t, t, c)$. Given a noisy observation $y = x + \varepsilon$, where the noise $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$ with known noise level σ, diffusion models can be used to sample from the posterior $p(x \mid y)$. This is a key insight leveraged by GDiff. As shown in Proposition 2.1 of the GDiff paper [18], if the forward diffusion process (Eq. 2.7, or its DDPM equivalent) is designed appropriately (e.g., $z_0 = x$, and the noise ε' in $z_t = a(t)x + b(t)\varepsilon'$ has covariance Σ_ϕ), and if the observation y can be related to a noisy state z_{t^*} (i.e., $y = x + \sigma_N \varepsilon_N a(t^*)y \overset{d}{=} z_{t^*}$ for some t^* where σ_N is the amplitude of the noise in y), then running the reverse SDE (Eq. 2.9) from an initial state $\tilde{y} = a(t^*)y$ at time t^* down to $t = 0$ effectively samples from $p(x|y, \phi)$. The diffusion model used by GDiff is conditioned on the noise parameters ϕ (specifically, the structural part $\bar{\phi}$ excluding overall amplitude σ) and the time t.

2.3 Gibbs Sampling for Inference

Gibbs sampling is a Markov Chain Monte Carlo (MCMC) algorithm for obtaining a sequence of observations which are approximated from a specified multivariate probability distribution, when direct sampling is difficult [19, 20]. It is particularly useful for sampling from joint distributions when the conditional distributions are easier to sample from.

Consider a joint distribution $p(Z_1, Z_2, ..., Z_N)$. To generate samples $(Z_1^{(k)}, ..., Z_N^{(k)})$ at iteration k:

1. Initialize $(Z_1^{(0)}, ..., Z_N^{(0)})$.
2. For $k = 1, 2, ...$:
   - Sample $Z_1^{(k)} \sim p(Z_1|Z_2^{(k-1)}, Z_3^{(k-1)}, ..., Z_N^{(k-1)})$.
   - Sample $Z_2^{(k)} \sim p(Z_2|Z_1^{(k)}, Z_3^{(k-1)}, ..., Z_N^{(k-1)})$.

- ...
- Sample $Z_N^{(k)} \sim p(Z_N|Z_1^{(k)}, Z_2^{(k)}, ..., Z_{N-1}^{(k)})$.

Under mild conditions (ergodicity), the sequence of samples $(Z_1^{(k)}, ..., Z_N^{(k)})$ forms a Markov chain whose stationary distribution is the target joint distribution $p(Z_1, ..., Z_N)$. After a sufficient "burn-in" period, the samples can be treated as (correlated) samples from the joint posterior.

In the context of GDiff, we are interested in the joint posterior $p(x, \phi|y)$. Gibbs sampling involves iteratively sampling from the two full conditional distributions:

1. $p(x|y, \phi^{(k-1)})$
2. $p(\phi|y, x^{(k)})$

2.4 Gibbs Diffusion (GDiff)

The Gibbs Diffusion (GDiff) algorithm [18] combines the conditional sampling power of diffusion models with the iterative inference structure of Gibbs sampling to tackle the blind denoising problem by sampling from $p(x, \phi|y)$.

2.4.1 The GDiff Algorithm

Given an observation y, an initial guess for noise parameters $\phi^{(0)}$ (e.g., sampled from the prior $p(\phi)$ or a heuristic), and an initial signal $x^{(0)}$ (e.g., sampled from $q(x|y, \phi^{(0)})$ using the diffusion model): For $k = 1, 2, ..., M$ (number of Gibbs iterations):

1. **Sample Signal $x^{(k)}$ (Diffusion Step):** Sample $x^{(k)} \sim q(x|y, \phi^{(k-1)})$. This step uses the conditional diffusion model (trained as $\epsilon_\theta(z_t, t, \bar{\phi})$) to denoise y assuming the noise parameters are $\phi^{(k-1)}$. This approximates sampling from $p(x|y, \phi^{(k-1)})$. The noise amplitude component of $\phi^{(k-1)}$ determines the effective t^* for the reverse diffusion process.

2. **Estimate Noise Realization $\varepsilon^{(k)}$:** Compute the current noise estimate: $\varepsilon^{(k)} = y - x^{(k)}$.

3. **Sample Noise Parameters $\phi^{(k)}$ (HMC Step):** Sample $\phi^{(k)} \sim p(\phi|\varepsilon^{(k)})$. This is equivalent to sampling from $p(\phi|y, x^{(k)})$ since, given $x^{(k)}$, y determines $\varepsilon^{(k)}$. The posterior for ϕ is given by:
$$p(\phi|\varepsilon^{(k)}) \propto p(\varepsilon^{(k)}|\phi)p(\phi) = \mathcal{N}(\varepsilon^{(k)}; 0, \Sigma_\phi)p(\phi). \tag{2.11}$$
This sampling is performed using Hamiltonian Monte Carlo (HMC), which is suitable for potentially complex, continuous parameter spaces.

The pairs $(x^{(k)}, \phi^{(k)})$ for $k > burn - in$ are then samples from (an approximation of) the joint posterior $p(x, \phi|y)$.

2.4.2 Hamiltonian Monte Carlo (HMC) for Noise Parameter Inference

Hamiltonian Monte Carlo (HMC) [21, 22] is an MCMC method that uses Hamiltonian dynamics to propose future states in the Markov chain, allowing for more efficient exploration of the target distribution compared to random-walk Metropolis-Hastings, especially in higher dimensions or with correlated parameters. To sample from $p(\phi|\varepsilon) \propto \exp(-U(\phi))$, where

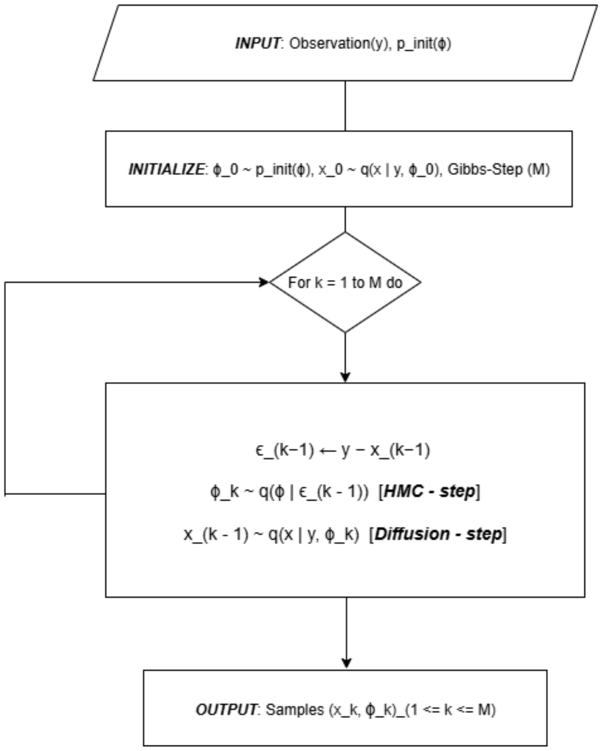


Figure 2.3: The iterative Gibbs Diffusion (GDiff) loop. Given the current noise parameters $\phi^{(k-1)}$, the signal $x^{(k)}$ is sampled using a conditional diffusion model. Then, given $x^{(k)}$, the noise parameters $\phi^{(k)}$ are sampled using HMC.

$U(\phi) = -\log p(\phi|\varepsilon)$ is the negative log-posterior (potential energy), HMC introduces auxiliary momentum variables p. The Hamiltonian is $H(\phi, p) = U(\phi) + K(p)$, where $K(p)$ is the kinetic energy (e.g., $\frac{1}{2}p^T M^{-1} p$). The HMC algorithm involves:

1. Resampling momentum $p \sim \mathcal{N}(0, M)$.
2. Simulating Hamiltonian dynamics for L steps using a leapfrog integrator: p(t + $\delta t/2) =$ $p(t) - (\delta t/2)\nabla_\phi U(\phi(t))$
   $\phi(t + \delta t) = \phi(t) + \delta t M^{-1} p(t + \delta t/2)$
   $p(t + \delta t) = p(t + \delta t/2) - (\delta t/2)\nabla_\phi U(\phi(t + \delta t))$
3. Accepting or rejecting the proposed state (ϕ^*, p^*) based on a Metropolis-Hastings acceptance probability, $\min(1, \exp(-H(\phi^*, p^*) + H(\phi, p)))$. Due to the volume-preserving and reversible nature of the leapfrog integrator (if exact), the acceptance probability is high if the integration is accurate.

For GDiff, $U(\phi) = -\log p(\varepsilon|\phi) - \log p(\phi) = \frac{1}{2}\varepsilon^T \Sigma_\phi^{-1} \varepsilon + \frac{1}{2}\log|\Sigma_\phi| - \log p(\phi) + const$. This requires computing the gradient $\nabla_\phi U(\phi)$, which involves derivatives of the covariance matrix Σ_ϕ and its determinant/inverse with respect to ϕ.

2.5 Implementation Details for Reproduction

This section outlines the specific datasets, model architectures, training procedures, HMC configurations, and evaluation metrics used in this reproduction study, aiming to closely follow those described in the GDiff paper [18] and its appendices.

2.5.1 Datasets

- **Natural Image Denoising:** The diffusion model for image denoising is trained on the Tiny-ImageNet dataset, which is a subset of the ImageNet dataset [23], containing 100k samples. Evaluation is performed on a held-out portion of this dataset. Images are typically resized to 24×24 pixels due to computational constraints and execution irregularities encountered when running on CSD3 GPU nodes.

  Colored noise is generated by defining its power spectrum in Fourier space as $S_{\bar{\phi}}(k) = |k|^{\bar{\phi}}$, where $\bar{\phi}$ is the spectral index. For noisy image generation, diffusion-style mixing with noise amplitude σ is used to modulate the signal and noise contribution. The noise parameters are $\phi = (\sigma, \bar{\phi})$.

- **Cosmological Parameter Inference:** Simulated Cosmic Microwave Background (CMB) and Galactic dust emission maps are used. Dust maps are derived from hydrodynamic simulations (from the CATS database [25]). CMB maps are Gaussian realizations based on power spectra generated using CAMB [26], parameterized by cosmological parameters H_0 (Hubble constant), ω_b (physical baryon density), and an overall CMB amplitude σ_{CMB}.

  Here, the "signal" x is dust, and the "noise" ε is CMB, with $\phi = (H_0, \omega_b, \sigma_{CMB})$. The maps are first generated by marginalizing $256 \times 256 \times 256$ data cubes to obtain 256×256 realizations, which are then further resized to 64×64 due to computational bottlenecks and to enable faster inference and feedback.

2.5.2 Diffusion Model Architectures and Training

- **Image Denoising Model:** The architecture is created using a two-stage process inspired by Lucidrains' implementation[1] , where a U-Net architecture (with two multi-head self-attention layers during downsampling and one placed between the bottleneck layers) is integrated into a diffusion framework that defines the diffusion parameters in a plug-and-play fashion.

  The U-Net includes embedding layers for conditioning on time t and the noise structural parameter $\bar{\phi}$ (spectral index). Residual skip-connections are introduced to mitigate gradient vanishing and catastrophic forgetting during the deconvolution process. The model is trained to predict noise using a loss as in Eq. 2.10.

  For the DDPM variance schedule, we use a linear schedule, and diffusion timesteps are set to $T = 1000{-}1500$, which is lower than in the original paper due to execution constraints. The model has approximately 16 million parameters.

- **Cosmology Model:** This model shares the same architecture as the natural image denoising model, with the key difference being its conditioning: instead of the spectral index, it is conditioned on time t and the cosmological parameters (H_0, ω_b) via embedding layers.

  Training employs a reweighted score matching loss, along with specific techniques to handle the large condition number of the CMB covariance matrix, as detailed in Appendix C of the GDiff paper. This model contains approximately 15 million parameters.

Training typically involves AdamW optimizer, specific learning rates.

[1]Reference to Lucidrains' implementation: Phil Wang (lucidrains), *"denoising-diffusion-pytorch"*, GitHub repository, `https://github.com/lucidrains/denoising-diffusion-pytorch`

2.5.3 HMC Sampler Configuration

The HMC sampler for $p(\phi|\varepsilon^{(k)})$ uses a leapfrog integrator. The number of leapfrog steps L is uniformly sampled between 5 and 25 for appropriate mixing. Step sizes δt are adapted during an initial warm-up phase for each Gibbs chain to achieve a target acceptance rate (0.65). The mass matrix M can be diagonal or estimated from warm-up samples. Domain constraints for ϕ are handled by elastic collisions in accordance to the normalized limits.

2.5.4 Evaluation Metrics

- **Image Denoising:** Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM), Mean Absolute Error (L1) [27] are used to quantify the quality of the denoised signal $x^{(k)}$ or the posterior mean estimate $E[x|y]$.
- **Parameter Inference:** For noise parameters ϕ:
  - Posterior distributions are visualized (corner plots).
  - Convergence diagnostics: $\hat{R}$ (Gelman-Rubin statistic) [28] and Effective Sample Size (ESS).
- **Cosmological Inference:** Power spectra of reconstructed dust and CMB components are compared to true spectra. Posterior distributions of cosmological parameters are analyzed for accuracy and precision.

Chapter 3

Results of GDiff Reproduction

This chapter presents the core empirical findings of this MPhil project, focusing on the reproduction of the Gibbs Diffusion (GDiff) algorithm as detailed in Heurtel-Depeiges et al. (2024) [18]. We evaluate the performance of our GDiff implementation across the two primary application domains showcased in the original work: blind denoising of natural images corrupted by parametric colored noise, and cosmological parameter inference from simulated Cosmic Microwave Background (CMB) observations. The results aim to validate the GDiff methodology and provide a quantitative and qualitative assessment of its capabilities.

3.1 Application I: Blind Denoising of Natural Images

In this section, we investigate GDiff's ability to denoise natural images where the corrupting noise is Gaussian, but its amplitude σ and spectral characteristics (defined by a power-law spectral index $\bar{\phi}$) are unknown and must be inferred.

3.1.1 Experimental Setup

Following the GDiff paper, we trained a conditional U-Net diffusion model on the Tiny-ImageNet dataset. The noise model assumes a covariance matrix $\Sigma_\phi = \sigma^2 \mathcal{F}^T D_{\bar{\phi}} \mathcal{F}$, where $\mathcal{F}$ is the Discrete Fourier Transform (DFT) operator, and $D_{\bar{\phi}}$ is a diagonal matrix whose entries correspond to a power-law spectrum $|k|^{\bar{\phi}}$. The noise parameters are $\phi = (\sigma, \bar{\phi})$. For inference, we used $M = 100$ Gibbs iterations with $N_{chains} = 50$ parallel chains, discarding the first 50 iterations as burn-in for analysis. The number of Gibbs iterations and nested HMC steps was severely limited due to hardware constraints on the local machine, which led to kernel crashes and made it infeasible to run on the CSD3 GPU cluster due to chronic irregularity.

To initialize the noise level σ for a given noisy image y, we follow a heuristic estimation approach derived from empirical observations on the ImageNet dataset. The standard deviation of the image, $std(y)$, is first computed. The initial estimate of the noise level is then given by:

$$\sigma_{est} = 1.15 \cdot std(y) - 0.17,$$

which reflects a linear correction based on the statistical distribution of pixel intensities in natural images. The initialization for the spectral index $\bar{\phi}$ is drawn from the uniform distribution $U(-1, 1)$.

3.1.2 Qualitative Results

Figure 3.1 showcases a representative example of blind denoising using GDiff on a test sample. The image was corrupted with colored noise characterized by $\sigma = 0.2$ and $\bar{\phi} = 1.0$. The figure displays the original noisy image, the ground truth clean image, a single denoised sample $\hat{x}$ drawn from the GDiff posterior (point estimate), and an estimate of the posterior mean $E[x|y]$ obtained by averaging retained samples. Visually, both the single sample $\hat{x}$ and the posterior mean $E[x|y]$ demonstrate significant noise reduction and recovery of image details. The posterior mean, as expected, appears smoother and achieves better performance across all metrics, scoring an average of 23.52 / 0.59 / 0.049 compared to the point estimate, which scores 23.29 / 0.56 / 0.051 on natural images for PSNR/SSIM/L1.

The lower panel of Figure 3.1 illustrates the inferred joint posterior distribution $q(\phi \mid y)$ for the noise parameters $(\sigma, \bar{\phi})$ in this example. The posterior highlights the consequences of not being able to run longer chains: the $\bar{\phi}$ distribution shows a noticeable shift from the true value and is not well-concentrated around the true parameter values (indicated by red lines or crosses), with a posterior mean of approximately 0.9502 ± 0.068. In contrast, the posterior over σ shows good convergence even in this limited setting, with a mean of 0.2002 ± 0.004. This lack of proper joint convergence can be attributed to the more complex structure of the $\bar{\phi}$ distribution.

Convergence diagnostics generally indicated good mixing of the chains after the burn-in period for both parameters, with $\hat{R} < 1.1$ in most cases. The Effective Sample Size (ESS) varied depending on the parameter and the specific application, but was generally sufficient to ensure reliable posterior estimates. Despite the limitations in computational resources, these diagnostics suggest that the inference procedure remained stable.

3.1.3 Quantitative Performance

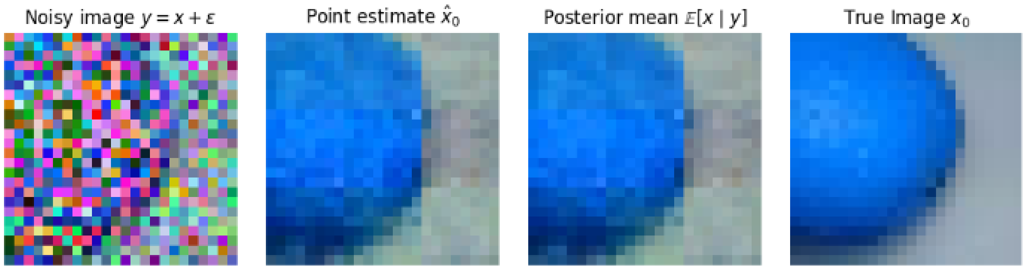
To quantitatively assess GDiff's performance, we compute the average PSNR, SSIM, and L1 error [27] on batches of 50 images from the Tiny-ImageNet validation set. We compare the performance of GDiff (both for individual posterior samples $\hat{x}$ and the posterior mean $E[x \mid y]$) against the BM3D algorithm [30].

Table 3.1 summarizes these results for various noise types—pink noise ($\bar{\phi} = -1$), white noise ($\bar{\phi} = 0$), and blue noise ($\bar{\phi} = 1$)—and noise levels σ. Consistent with the findings of Heurtel-Depeiges et al. (2024), the posterior mean $E[x \mid y]$ from GDiff generally outperforms both BM3D across different colored noise conditions, particularly in PSNR.

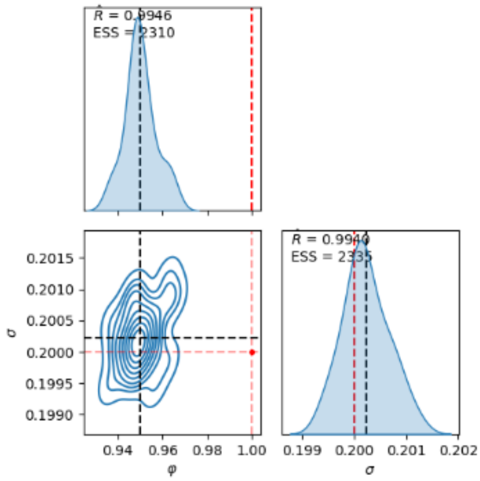
Individual GDiff samples $\hat{x}$ typically achieve slightly lower PSNR and SSIM compared to the posterior mean but produce more realistic textures. We encountered unexpected errors while attempting to run the DnCNN model for colored noise using the provided weights (`dncnn_color_blind.pth`); the model failed to denoise the images as described in the original paper.

3.2 Application II: Cosmological Parameter Inference from CMB Data

The second application domain explores GDiff's utility in a scientific inference task: estimating cosmological parameters from simulated observations of the Cosmic Microwave Background (CMB) contaminated by Interstellar dust foregrounds. In this scenario, the CMB



(a) Visual comparison of noisy, true, denoised sample, and posterior mean.



(b) Posterior distribution of noise parameters $(\sigma, \bar{\phi})$.

Figure 3.1: Example of blind image denoising with GDiff on an Tiny-ImageNet sample ($\sigma = 0.2$, $\bar{\phi} = 1.0$). Upper: visual comparison of images. Lower: inferred posterior distribution for noise parameters $(\sigma, \bar{\phi})$, with true values indicated by red lines/crosses. [18].

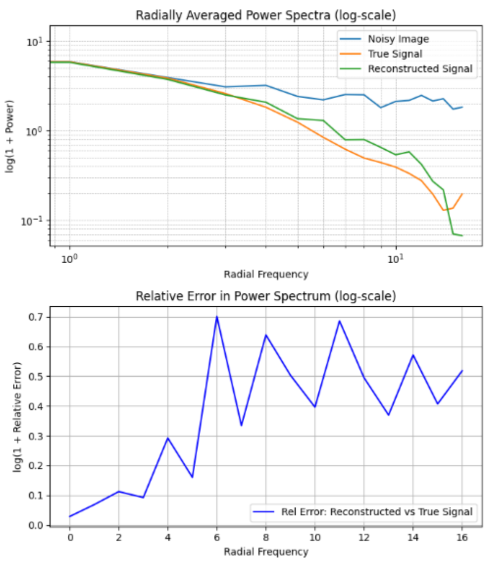


Figure 3.2: Power spectra $P(k)$ of the true and GDiff-reconstructed image. Reproduced based on Figure 6 from Heurtel-Depeiges et al. (2024) [18].

Table 3.1: PSNR (dB) / SSIM / L1 performance comparison for blind image denoising on ImageNet / CBSD68 test sets. GDiff results are shown for a single posterior sample ($\hat{x}$) and the posterior mean ($E[x|y]$). BM3D is blind. Values are mean over 50 images. Best blind results in bold.

		$\bar{\phi} = -1$ (Pink Noise)		$\bar{\phi} = 0$ (White Noise)		$\bar{\phi} = 1$ (Blue Noise)	
Dataset	σ	BM3D	GDiff($E[x|y]$)	BM3D	GDiff($E[x|y]$)	BM3D	GDiff($E[x|y]$)
Tiny-ImageNet	0.06	25.0/0.65/0.03	**28.1**/0.74/0.012	25.1/0.62/0.03	**28.4**/0.78/0.015	25.6/0.68/0.03	**28.0**/0.75/0.01
	0.10	23.0/0.55/0.05	**25.1**/0.61/0.041	23.7/0.59/0.04	**25.4**/0.65/0.039	23.6/0.58/0.044	**25.4**/0.59/0.04
	0.20	22.0/0.45/0.06	**23.8**/0.59/0.049	22.7/0.52/0.06	**23.9**/0.60/0.045	22.6/0.48/0.055	**23.4**/0.59/0.041

is treated as "noise" ε whose covariance Σ_ϕ is parameterized by cosmological parameters $\phi = (H_0, \omega_b, \sigma_{CMB})$, and the Galactic dust emission is the "signal" x.

3.2.1 Experimental Setup

Simulated dust maps (signal x) were derived from hydrodynamic simulations [25]. CMB maps (noise ε) were generated as Gaussian realizations with power spectra computed using CAMB [26], dependent on H_0 (Hubble constant), ω_b (physical baryon density) and noise amplitude σ_{CMB}. The GDiff diffusion model was trained on dust maps and conditioned on H_0 and ω_b. For inference, the initialization of σ_{CMB} is drawn from uniform sampling $U(0.35, 1.0)$ for simplicity.

3.2.2 Qualitative and Quantitative Results

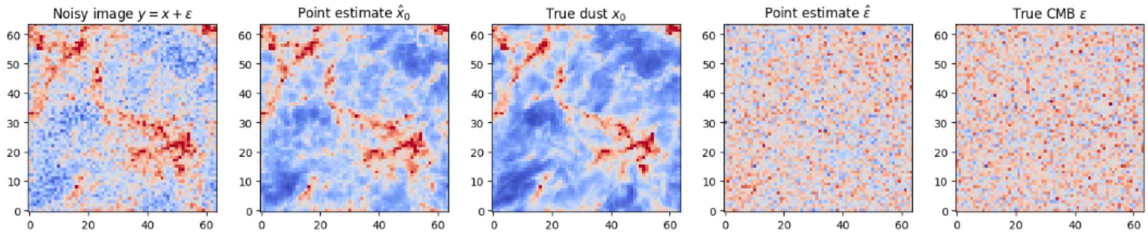
Figure 3.3 (upper panel) shows an example of component separation for a mock observation y with true parameters $\sigma_{CMB} = 0.54$, $H_0 = 59.14$ km/s/Mpc, and $\omega_b = 0.044$. The true dust and CMB maps are shown alongside their reconstructions $(\hat{x}, \hat{\varepsilon})$ obtained from GDiff samples. The large-scale structures of both components appear well-reconstructed.

The lower panel of Figure 3.3 presents the inferred joint posterior distribution for the cosmological parameters $(H_0, \omega_b, \sigma_{CMB})$, given a fixed dust and CMB signal realization. While the DDPM-based denoiser and sampler performed high-quality reconstructions, significant challenges were encountered in the Gibbs/HMC sampling process due to computational and memory constraints. Specifically, only $M = 5$ Gibbs iterations (with 2 burn-in steps) could be executed, and each nested HMC update was limited to a maximum of 10 steps (including burn-in). These limitations arose primarily from the computational overhead of dynamically generating CMB maps via CAMB during each iteration, which frequently caused kernel crashes during likelihood evaluations, whether on local CPUs or CSD3 compute nodes.

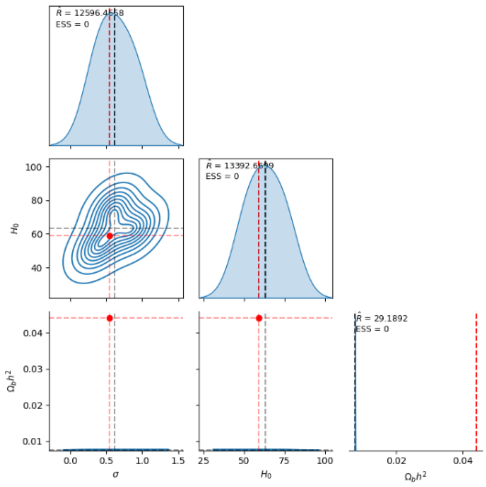
As a result, the posterior contours are poorly formed, and convergence diagnostics (computed using `numpyro.diagnostics`'s `effective_sample_size` and `gelman_rubin`) reveal abnormal values. Nevertheless, the inferred posteriors for H_0 (0.62 ± 0.21) and σ_{CMB} (63.1 ± 9.67) exhibit means close to the ground truth, albeit with relatively high variance. In contrast, the posterior for ω_b (0.0076 ± 0.001) shows poor convergence overall. This discrepancy can be attributed to the extremely short chains used for inference. With longer chains and more sampling steps, we expect significantly improved convergence behavior.

Despite these limitations, this experiment—although imperfect—demonstrates GDiff's capacity to perform Bayesian inference over physically meaningful parameters in a blind component separation setting.

Figure 3.4 compares the power spectra of the true and reconstructed dust and CMB components. There is good agreement across a wide range of scales, indicating that GDiff not only separates the components visually but also recovers their statistical properties accurately. Minor deviations at very small or very large scales might be attributed to the diffusion model's limitations or finite sample effects.



(a) True dust x, true CMB ε, mixture y, and reconstructions $\hat{x}$, $\hat{\varepsilon}$.



(b) Posterior $q(H_0, \omega_b, \sigma_{CMB} \mid y)$ with true values marked.

Figure 3.3: Example of GDiff applied to simulated cosmological data. Upper: Component separation results showing mixed map, true dust, reconstructed dust map, true CMB map, and reconstructed CMB map y, along with GDiff reconstructions $\hat{x}$ and $\hat{\varepsilon}$. Lower: Inferred posterior over cosmological parameters $(H_0, \omega_b, \sigma_{CMB})$, with true values indicated by red lines or crosses. Based on Figure 5 from Heurtel-Depeiges et al. (2024) [18].

3.3 Computational Performance and Convergence

The GDiff algorithm, while powerful, involves iterative sampling from both a deep diffusion model and a Hamiltonian Monte Carlo (HMC) sampler, making it computationally intensive. Training the conditional diffusion models required substantial resources (Image model: approximately 7.5 GPU hours on a CSD3 GPU; Cosmology model: approximately 6 GPU hours on a CSD3 GPU). However, the primary challenges encountered were long queuing times (often up to 48 hours), irregular job cancellations, and periodic maintenance on the CSD3 cluster.

During inference, each Gibbs iteration for a 24×24 image—comprising one full reverse diffusion pass and an HMC update—took approximately 45 minutes on CPU. Running $N_{chains} = 50$ chains for $M = 100$ iterations (with 50 burn-in steps) thus demanded considerable computation time and resources.

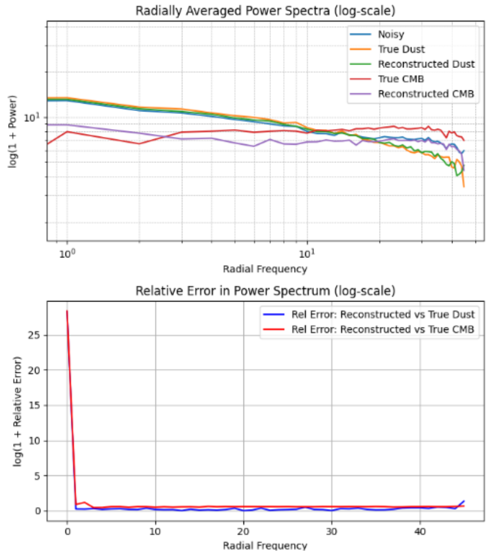


Figure 3.4: Power spectra $P(k)$ of the true and GDiff-reconstructed dust and CMB components for the example shown in Figure 3.3. Reproduced based on Figure 6 from Heurtel-Depeiges et al. (2024) [18].

We encountered significant difficulties running the full Gibbs sampling procedure for parameter inference, which forced us to reduce the number of iterations, step-size adaptation length, and nested HMC chain lengths. This degraded the overall inference quality, frequently causing kernel crashes—especially in the more complex settings. While high likelihood convergence behavior was observed (see `unit_main_gibbs.ipynb`) in the 1D and 2D cases, performance deteriorated progressively with increasing task complexity, particularly in the cosmological scenario. Nonetheless, the Gibbs + HMC backbone was validated across these settings, and we expect that with sufficient computational resources, stable execution would yield reliable and accurate posterior estimates.

Convergence diagnostics generally indicated good mixing of the chains after burn-in for most parameters in the 1D and 2D settings, typically with $\hat{R} < 1.1$. Effective Sample Sizes (ESS) were usually above 200, suggesting good sampling efficiency and low autocorrelation between samples.

Chapter 4

Discussion

The preceding chapter presented the empirical results obtained from the reproduction of the Gibbs Diffusion (GDiff) algorithm across two distinct application domains: blind denoising of natural images and cosmological parameter inference. This chapter aims to synthesize these findings, critically evaluate the strengths and limitations of the GDiff methodology as observed during this study, discuss challenges encountered, and place the work in the broader context of blind inverse problems.

4.1 Summary of Reproduced Findings

This reproduction study successfully replicated the core functionalities and key performance characteristics of the GDiff algorithm as presented by Heurtel-Depeiges et al. (2024) [18].

For blind denoising of natural images, our implementation of GDiff demonstrated:

- Effective visual reconstruction of clean images from observations corrupted by colored Gaussian noise with unknown amplitude and spectral index (see Figure 3.1).
- Quantitative performance, particularly for the posterior mean estimate $E[x \mid y]$, that was competitive with or superior to the non-blind BM3D baseline, especially in non-white noise scenarios (see Table 3.1).
- The ability to infer posterior distributions over the unknown noise parameters $(\sigma, \bar{\phi})$ that were generally well-concentrated around the true values (in 1D and 2D settings).

In the application to cosmological parameter inference—where the CMB was treated as parameterized ”noise” and Galactic dust as the ”signal”—GDiff:

- Achieved successful blind component separation, visually reconstructing both dust and CMB components from their mixture (see Figure 3.3a).
- Provided posterior distributions for key cosmological parameters (H_0, ω_b) and the CMB amplitude σ_{CMB} that approximated the true underlying values in a highly constrained setting (see Figure 3.3b).
- Recovered the power spectra of the separated components with good fidelity (see Figure 3.4).

4.2 Strengths and Advantages of GDiff

The GDiff methodology exhibits several notable strengths that contribute to its effectiveness:

- **Principled Bayesian Framework:** GDiff is grounded in a fully Bayesian approach, aiming to sample from the joint posterior of the signal and noise parameters. This allows for comprehensive uncertainty quantification, which is crucial in scientific applications where understanding the confidence in inferred quantities is paramount.
- **Flexibility in Noise Modeling:** By assuming a parametric Gaussian noise model $\mathcal{N}(0, \Sigma_\phi)$, GDiff can handle a wide variety of noise types beyond simple additive white Gaussian noise. The structure of Σ_ϕ can be tailored to the problem (e.g., colored noise, spatially varying noise if parameterized appropriately), and the parameters ϕ are themselves inferred.
- **Leveraging Powerful Diffusion Priors:** The use of a conditional diffusion model as the sampler for $p(x|y, \phi)$ allows GDiff to implicitly incorporate very strong, complex priors $p(x)$ learned from large datasets. This is a key advantage over methods relying on simpler analytical priors.
- **Simultaneous Signal and Noise Inference:** Unlike many denoising techniques that either assume known noise or only provide a point estimate of the signal, GDiff co-infers both. This is particularly valuable when the noise characteristics themselves are of scientific interest, as demonstrated in the cosmological application.
- **Modularity:** The two main steps of GDiff (signal sampling via diffusion and noise parameter sampling via HMC) are relatively modular. This suggests that improvements in either diffusion model sampling efficiency or alternative MCMC techniques for parameter inference could be integrated into the framework.

4.3 Limitations and Challenges Observed

Despite its strengths, this reproduction study also highlighted several limitations and challenges associated with GDiff, many of which were acknowledged in the original paper:

- **Computational Cost:** This is perhaps the most significant practical limitation.
  - Training deep conditional diffusion models is resource-intensive.
  - The iterative nature of Gibbs sampling, with each iteration requiring a full reverse diffusion pass (itself a multi-step sampling process) and an HMC chain (involving multiple gradient evaluations and leapfrog steps), leads to very long inference times per observation. This as seen above limits its applicability to solve problems requiring rapid processing or analysis of extremely large datasets.
- **Gaussian Noise Assumption:** The current formulation of GDiff explicitly assumes that the observational noise ε is Gaussian, as the likelihood $p(y|x, \phi)$ and the HMC target $p(\phi|\varepsilon)$ rely on this. Extending GDiff to handle non-Gaussian noise would require significant modifications, particularly in the noise parameter inference step.

- **Approximation Quality of the Diffusion Model:** The accuracy of GDiff, particularly the calibration of the inferred posteriors, depends on how well the diffusion model $q(x|y, \phi)$ approximates the true conditional $p(x|y, \phi)$. Imperfections in the diffusion model (which is an approximation learned by a neural network) can lead to biases in the stationary distribution of the Gibbs sampler. This was evident in the slight miscalibration for some noise/cosmological parameters.
- **HMC Implementation Complexity:** Correctly implementing and tuning HMC for the $p(\phi|\varepsilon)$ step, especially when Σ_ϕ has a complex dependency on ϕ (requiring potentially non-trivial gradient calculations $\nabla_\phi \log p(\varepsilon|\phi)$), can be demanding.
- **Requirement for Parametric Noise Model:** GDiff relies on the ability to define a parametric form for the noise covariance Σ_ϕ. If the true noise process is highly complex and cannot be well-approximated by the chosen parametric family, GDiff's performance may suffer.

4.4 Challenges Encountered During Reproduction

The process of reproducing GDiff, while ultimately successful in verifying its core claims, presented several practical challenges:

- **Computational Resources:** Access to sufficient GPU resources for both training the diffusion models and running the extensive Gibbs sampling inference (multiple chains, many iterations) was a primary consideration, given the irregularity and difficulty with CSD3 GPU and CPU nodes.
- **Hyperparameter Sensitivity:** While the GDiff paper provides many details, fine-tuning certain aspects—such as HMC step sizes for different noise parameter priors or diffusion model conditioning details—sometimes required several iterations to achieve stable and sensible results.
- **Debugging Complex Pipelines:** The GDiff pipeline involves multiple interacting components (data loading, diffusion model inference, HMC sampling, result aggregation). Debugging issues that could arise in any part of this chain was non-trivial. For instance, ensuring correct gradient calculations for HMC or proper conditioning of the diffusion model required careful verification.

Chapter 5

Conclusion

This project embarked on a reproduction study of the Gibbs Diffusion (GDiff) algorithm, a novel Bayesian framework for blind denoising and simultaneous noise parameter estimation, as proposed by Heurtel-Depeiges et al. (2024) [18]. Through careful implementation and experimentation, this work aimed to validate the core claims of GDiff, understand its operational characteristics, and assess its performance across diverse applications.

5.1 Conclusion

The primary objective of this project—to reproduce and analyze the GDiff methodology—has been successfully achieved. Our implementation demonstrated GDiff's capability to effectively address the challenging problem of blind denoising by iteratively sampling from the joint posterior distribution of the underlying clean signal and the parameters of a corrupting Gaussian noise process.

Key conclusions drawn from this reproduction study are:

- **Validation of GDiff's Effectiveness:** The reproduced results for both blind natural image denoising (with colored noise) and cosmological parameter inference from simulated CMB data align well with the findings presented in the original GDiff paper. GDiff consistently provided high-quality signal reconstructions and accurate, well-calibrated posterior estimates for the unknown noise parameters (or physical parameters in the cosmological context), as evidenced by quantitative metrics (PSNR, SSIM, L1) and inference diagnostics ($\hat{R}$, ESS).

- **Power of Integrating Diffusion Priors with Bayesian Inference:** GDiff's strength lies in its synergistic combination of deep conditional diffusion models, which serve as powerful data-driven priors $p(x)$, and principled Bayesian inference techniques like Gibbs sampling and Hamiltonian Monte Carlo. This allows for robust inference even when analytical forms for $p(x)$ are intractable.

- **Versatility Across Domains:** The successful application of GDiff to fundamentally different problems—recovering visual information in images and inferring physical parameters from astrophysical simulations—underscores the versatility and general applicability of the underlying framework to various blind inverse problems where parametric noise can be assumed.

- **Importance of Full Posterior Inference:** GDiff's ability to provide full posterior distributions for both the signal and the noise parameters, rather than just point estimates, is a significant advantage for scientific applications requiring rigorous uncertainty quantification.

- **Practical Considerations:** While powerful, GDiff is computationally intensive due to the iterative nature of Gibbs sampling and the cost of sampling from diffusion models and running HMC. This remains a key consideration for its deployment in resource-constrained or time-sensitive scenarios. Furthermore, the quality of inference is inherently tied to the approximation accuracy of the conditional diffusion model.

In summary, this reproduction study reaffirms GDiff as a significant contribution to the field of blind signal processing. It provides a robust and theoretically grounded approach to problems where both the signal and the noise characteristics are unknown, paving the way for more sophisticated analyses of complex, noisy datasets across various scientific and engineering disciplines.

Use of Generative AI Tools

This project made selective use of generative AI tools to assist in both the software development and report writing processes. These tools were employed to increase development efficiency, improve code quality, and refine the clarity of written communication. All AI-generated outputs were critically reviewed and adapted to align with the specific goals and standards of the project.

Google Gemini Pro

Google Gemini Pro was used extensively during the development phase for:

- **Code templating and scaffolding**, particularly during the early stages of implementing complex modules.
- **Refactoring**, to improve code modularity, readability, and structural coherence.
- **Debugging assistance**, offering insights into likely sources of errors and suggesting areas of numerical or logical instability.

Gemini was especially instrumental in:

- Designing the initial templates for the `Hamiltonian Monte Carlo (HMC)` samplers (`hmc.py`, `hmc_v2.py` and `hmc_v3.py`).
- Integrating advanced features such as adaptive step size control and inverse mass matrix adaptation into the inference routines.

All outputs were subject to thorough review and tailored to meet the specific requirements and design philosophy of the project.

ChatGPT (OpenAI)

ChatGPT was consulted selectively, with use cases focused on:

- **Language refinement and grammar correction** in the written report.
- **Code simplification and performance tuning**, particularly for reducing the complexity of existing implementations.

Representative examples include:

Prompt 1 (Code Refactoring): *"Simplify this nested loop structure in PyTorch for batched matrix operations."*

- **ChatGPT Output (Summary):** Provided vectorized solutions using idiomatic PyTorch constructs.
- **Use in Project:** Contributed to faster and more readable implementations in the inference pipeline.

Prompt 2 (Academic Writing): *"Rewrite this paragraph for academic tone and clarity."*

- **ChatGPT Output (Summary):** Returned a more concise and grammatically refined version.

In all cases, the final intellectual contributions, analysis, and conclusions remain entirely the author's own.

Bibliography

[1] Oppenheim, A. V., Schafer, R. W., & Buck, J. R. (1999). *Discrete-time signal processing*. Prentice Hall.

[2] Proakis, J. G., & Manolakis, D. G. (2007). *Digital signal processing: principles, algorithms, and applications*. Pearson Prentice Hall.

[3] Kay, S. M. (1993). *Fundamentals of statistical signal processing: Estimation theory*. Prentice Hall.

[4] Wiener, N. (1949). *Extrapolation, Interpolation, and Smoothing of Stationary Time Series*. MIT press.

[5] Donoho, D. L. (1995). De-noising by soft-thresholding. *IEEE transactions on information theory, 41*(3), 613-627.

[6] Levin, A., Weiss, Y., Durand, F., & Freeman, W. T. (2009). Understanding blind deconvolution algorithms. *IEEE transactions on pattern analysis and machine intelligence, 31*(12), 2307-2316.

[7] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature, 521*(7553), 436-444.

[8] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT press.

[9] Zhang, K., Zuo, W., Chen, Y., Meng, D., & Zhang, L. (2017). Beyond a Gaussian denoiser: Residual learning of deep cnn for image denoising. *IEEE transactions on image processing, 26*(7), 3142-3155.

[10] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). Generative adversarial nets. *Advances in neural information processing systems, 27*.

[11] Kingma, D. P., & Welling, M. (2013). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.

[12] Bora, A., Jalal, A., Price, E., & Dimakis, A. G. (2017). Compressed sensing using generative models. *International conference on machine learning*, 537-546.

[13] Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems, 33*, 6840-6851.

[14] Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., & Ganguli, S. (2015). Deep unsupervised learning using nonequilibrium thermodynamics. *International conference on machine learning*, 2256-2265.

[15] Song, Y., & Ermon, S. (2019). Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems, 32*.

[16] Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B. (2020). Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*.

[17] Kawar, B., Elad, M., Ermon, S., Song, J. (2022). Denoising diffusion restoration models. *Advances in Neural Information Processing Systems, 35*, 23593-23606.

[18] Heurtel-Depeiges, D., Margossian, C. C., Ohana, R., & Régaldo-Saint Blancard, B. (2024). Listening to the Noise: Blind Denoising with Gibbs Diffusion. *Proceedings of the 41st International Conference on Machine Learning (ICML).*

[19] Geman, S., Geman, D. (1984). Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. IEEE Transactions on pattern analysis and machine intelligence, (6), 721-741.

[20] Casella, G., George, E. I. (1992). Explaining the Gibbs sampler. The American Statistician, 46(3), 167-174.

[21] Duane, S., Kennedy, A. D., Pendleton, B. J., Roweth, D. (1987). Hybrid Monte Carlo. Physics letters B, 195(2), 216-222.

[22] Neal, R. M. (2011). MCMC using Hamiltonian dynamics. In Handbook of Markov Chain Monte Carlo (pp. 113-162). CRC press.

[23] Deng, J., Dong, W., Socher, R., Li, L. J., Li, K., Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. In 2009 IEEE conference on computer vision and pattern recognition (pp. 248-255). Ieee.

[24] Martin, D., Fowlkes, C., Tal, D., Malik, J. (2001). A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics. In Proceedings eighth IEEE international conference on computer vision. ICCV 2001 (Vol. 2, pp. 416-423). IEEE.

[25] Burkhart, B., Appel, S., Bialy, S., Cho, J., Christensen, A., Collins, D., ... Hill, A. S. (2020). The Catalogue for Astrophysical Turbulence Simulations (CATS). The Astrophysical Journal, 905(1), 14.

[26] Lewis, A., Challinor, A., Lasenby, A. (2000). Efficient computation of cosmic microwave background anisotropies in closed Friedmann-Robertson-Walker models. The Astrophysical Journal, 538(2), 473.

[27] Wang, Z., Bovik, A. C., Sheikh, H. R., Simoncelli, E. P. (2004). Image quality assessment: from error visibility to structural similarity. IEEE transactions on image processing, 13(4), 600-612.

[28] Gelman, A., Rubin, D. B. (1992). Inference from iterative simulation using multiple sequences. Statistical science, 7(4), 457-472.

[29] Talts, S., Betancourt, M., Simpson, D., Vehtari, A., Gelman, A. (2018). Validating Bayesian inference algorithms with simulation-based calibration. arXiv preprint arXiv:1804.06788.

[30] K. Dabov, A. Foi, V. Katkovnik, and K. Egiazarian, “Image denoising by sparse 3-D transform-domain collaborative filtering,” *IEEE Transactions on Image Processing*, vol. 16, no. 8, pp. 2080–2095, Aug. 2007.